End-to-End Learned Communication System: Joint Optimization of Constellation and Receiver over an AWGN Channel

A Project Report

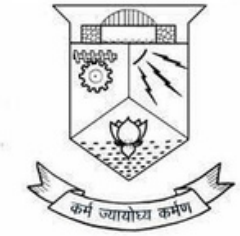
Submitted as part of the course PCECT502– Analog and Digital Communication

B.Tech. in Electronics and Communication Engineering

APJ Abdul Kalam Technological University

by

12 Akshay M Rahul
27 Dhanush Krishna R N
52 Pranav S



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
COLLEGE OF ENGINEERING TRIVANDRUM
KERALA
October 2026

Abstract

This project studies whether small neural networks can learn to act as digital receivers for AWGN-corrupted modulated symbols, and whether an end-to-end autoencoder can discover a useful signal constellation on its own. The report has two parts. The first examines a Gray-mapped QPSK receiver, a two-layer multilayer perceptron (MLP) trained with binary cross-entropy. Its bit error rate (BER) is evaluated under two setups at $E_b/N_0 = 5\,\text{dB}$ and across a sweep from 0 to 10 dB, and compared with the closed-form QPSK BER. The second part trains a 16-ary autoencoder with a neural encoder, a power normalization layer, an AWGN channel, and a neural decoder at 7 dB, and examines the learned two-dimensional constellation. The learned QPSK receiver tracks the theoretical curve closely across the sweep, with deviations at high E_b/N_0 that are consistent with Monte Carlo sampling limits. The autoencoder converges to a non-uniform 16-point constellation with an outer ring, an inner ring, and a point near the origin. The report discusses the results, the assumptions behind them, and several caveats.

Contents

1 Introduction **3**

2 Background **3**
- 2.1 QPSK and Gray Mapping . . . 3
- 2.2 AWGN Channel and Noise Scaling . . . 3
- 2.3 Theoretical QPSK Bit Error Rate . . . 3
- 2.4 Autoencoder-Based Communication . . . 3

3 Methodology **4**
- 3.1 QPSK Receiver: Design and Training . . . 4
- 3.2 QPSK Evaluation at 5 dB . . . 4
- 3.3 QPSK BER Sweep over Eb/N0 . . . 4
- 3.4 16-ary Autoencoder Constellation Design . . . 4

4 Results **5**
- 4.1 QPSK Receiver Performance at 5 dB . . . 5
- 4.2 QPSK BER Across Eb/N0 . . . 6
- 4.3 Learned 16-ary Constellation . . . 7

5 Discussion **8**
- 5.1 Learned QPSK Receivers Match Theory . . . 8
- 5.2 Equivalence of the Two Setups . . . 8
- 5.3 The Autoencoder Discovers a Non-Uniform Constellation . . . 8
- 5.4 Normalization Convention and Effective SNR . . . 8
- 5.5 Limitations . . . 9

6 Conclusion **9**

A QPSK Code at 5 dB **10**

B BER Code: Learned QPSK Receiver over Eb/N0 **11**

C Autoencoder Code: 16-ary Learned Constellation **12**

1 Introduction

Conventional digital communication systems are designed around analytically tractable components: a modulator maps bits to points in a signal space, the channel adds noise, and a receiver uses a known decision rule. Deep learning offers an alternative in which the receiver, and even the transmitter, is learned from data. This is attractive when the channel is difficult to model analytically, but it also raises a basic question: does a learned system recover the performance that theory predicts in a case where theory is available?

This report answers that question in two parts. For QPSK, where the bit error rate of an optimal receiver is known in closed form, we check whether a neural decoder reaches that performance. For a 16-point constellation, where no simple closed-form optimum is readily available, we let an autoencoder design the signal set and inspect the result. The report is organized as follows. Section 2 reviews the relevant background. Section 3 describes the methods. Section 4 presents the results. Section 5 discusses their meaning and limitations, and Section 6 concludes. The source code is reproduced in the appendices.

2 Background

2.1 QPSK and Gray Mapping

Quadrature phase-shift keying (QPSK) transmits two bits per symbol. Each pair of bits $(b_0, b_1) \in \{0,1\}^2$ is mapped to a point in the in-phase/quadrature (I/Q) plane. The Gray mapping used here is

$$I = \frac{2b_0 - 1}{\sqrt{2}}, \qquad Q = \frac{2b_1 - 1}{\sqrt{2}}, \tag{1}$$

so that each symbol has unit average energy $E_s = 1$ and neighbouring constellation points differ in only one bit. Because $E_b = E_s/2$ for QPSK, the bit energy is $E_b = 0.5$.

2.2 AWGN Channel and Noise Scaling

The additive white Gaussian noise (AWGN) channel returns $\mathbf{y} = \mathbf{x} + \mathbf{n}$, where $\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_2)$ acts independently on each real dimension. For a target E_b/N_0 with linear value $\gamma_b = 10^{(E_b/N_0 \text{ in dB})/10}$, the noise spectral density is $N_0 = E_b/\gamma_b$, and the per-dimension noise variance is $\sigma^2 = N_0/2$. In the code, this is implemented as

$$\sigma = \frac{0.5}{\sqrt{\gamma_b}}, \tag{2}$$

which is consistent with $\sigma^2 = N_0/2 = 0.25/\gamma_b$ when $E_b = 0.5$.

2.3 Theoretical QPSK Bit Error Rate

With Gray mapping, QPSK with coherent detection has the same bit error probability as BPSK with the same E_b/N_0:

$$P_b^{\text{QPSK}} = Q\left(\sqrt{\frac{2E_b}{N_0}}\right) = \frac{1}{2}\operatorname{erfc}\left(\sqrt{\frac{E_b}{N_0}}\right), \tag{3}$$

where $Q(\cdot)$ is the Gaussian tail function and erfc the complementary error function. This closed form is the benchmark used throughout the report.

2.4 Autoencoder-Based Communication

An autoencoder communication system treats the transmitter and receiver as neural networks trained jointly. The encoder maps a one-hot message $\mathbf{m} \in \{0,1\}^M$ to a real vector in $\mathbb{R}^n$, which is then normalized to satisfy a power constraint and transmitted over the channel. The decoder

produces logits over the M messages, and training minimizes the cross-entropy between these logits and the true message. The learned encoder output is the signal constellation.

3 Methodology

3.1 QPSK Receiver: Design and Training

The QPSK receiver is a decoder network

$$f_\theta : \mathbb{R}^2 \to \mathbb{R}^2, \qquad f_\theta = \text{Linear}(2, 64) \;\to\; \text{ReLU} \;\to\; \text{Linear}(64, 2), \tag{4}$$

which takes a noisy I/Q sample and outputs two logits, one per bit. Each bit is trained as an independent binary classification problem using `BCEWithLogitsLoss`. Training uses Adam with learning rate 10^{-3}, batch size 1024, and 5000 steps, with fresh random bits in each batch so that the training data is effectively unlimited. This same receiver design is used in both of the QPSK evaluations below.

3.2 QPSK Evaluation at 5 dB

The first evaluation fixes the operating point at $E_b/N_0 = 5\,\text{dB}$ and tests the trained receiver under two setups:

- **Setup 1 (random bits through the modulator).** Random bits are encoded to QPSK, corrupted by AWGN, and decoded. The BER is computed over 10^5 bits.
- **Setup 2 (AWGN applied directly to codewords).** A message index is drawn uniformly from the four 2-bit messages, the corresponding clean QPSK codeword is selected, AWGN is added, and the trained decoder is applied. Statistically this is the same channel as Setup 1, but the code path differs, which provides a consistency check on the implementation.

The full code is listed in Appendix A.

3.3 QPSK BER Sweep over Eb/N0

The second evaluation extends the receiver test to a sweep over $E_b/N_0 \in \{0, 1, \ldots, 10\}\,\text{dB}$. For each operating point the decoder is re-initialized, trained for 5000 steps with the same hyperparameters, and evaluated on 10^6 test bits. The learned BER is compared to the closed-form expression above. Because a zero error count cannot be displayed on a logarithmic axis, the plotted learned BER is floored at 10^{-7}. The full code is listed in Appendix B.

3.4 16-ary Autoencoder Constellation Design

The second part of the report learns a 16-point, two-dimensional constellation. The architecture is:

1. **Encoder:** a one-hot input of length 16 passes through Linear(16, 32), ReLU, and Linear(32, 2), producing a real 2-vector.
2. **Power normalization:** the encoder outputs are rescaled so that the mean squared norm over the batch equals $n_{\text{dim}} = 2$.
3. **Channel:** AWGN with per-dimension variance $\sigma^2 = 1/(2\,\text{SNR})$, where $\text{SNR} = 10^{\text{SNR}_{\text{dB}}/10}$ and $\text{SNR}_{\text{dB}} = 7$.
4. **Decoder:** Linear(2, 32), ReLU, Linear(32, 16), producing 16 logits.

The network is trained with cross-entropy against the true message index, using Adam with learning rate 10^{-2}, batch size 512, 200 epochs, and 100 steps per epoch (20,000 gradient steps in total). After training, the encoder is evaluated on the 16 one-hot inputs to extract the learned constellation. A separate sweep evaluates the symbol error rate (SER) over SNR $\in \{0, 2, \ldots, 12\}$ dB using 10^4 random messages per point. The full code is listed in Appendix C.

4 Results

4.1 QPSK Receiver Performance at 5 dB

Table 1 reports the BER for the two setups at $E_b/N_0 = 5$ dB, alongside the theoretical value. The theoretical value is computed as $\frac{1}{2}\operatorname{erfc}(\sqrt{\gamma_b})$ with $\gamma_b = 10^{0.5}$, giving approximately 5.94×10^{-3}.

Table 1: BER at $E_b/N_0 = 5$ dB for the two evaluation setups with a 100,000-bit test set.

Method	BER	Difference from theory
QPSK theory (closed form)	5.94×10^{-3}	—
Learned decoder, Setup 1	5.69×10^{-3}	-2.5×10^{-4}
Learned decoder, Setup 2	5.85×10^{-3}	-9×10^{-5}

Both setups fall within about 4% of the theoretical value. With 10^5 bits and a true BER near 6×10^{-3}, the standard error of the BER estimate is

$$\sigma_{\hat{p}} = \sqrt{\frac{p(1-p)}{N}} \approx \sqrt{\frac{0.006 \times 0.994}{10^5}} \approx 2.4 \times 10^{-4}, \tag{5}$$

so both measured values lie within about one standard error of theory. The gap between Setup 1 and Setup 2 (1.6×10^{-4}) is also less than one standard error, which is consistent with the two setups being statistically equivalent.

Figure 1 shows the received samples for each setup, coloured by the decoded bit pair. In both panels the decoder partitions the plane into four quadrants separated by the axes, which is the decision rule expected for Gray QPSK with equal priors. The learned decoder therefore recovers the coordinate-axis decision boundaries from data alone.

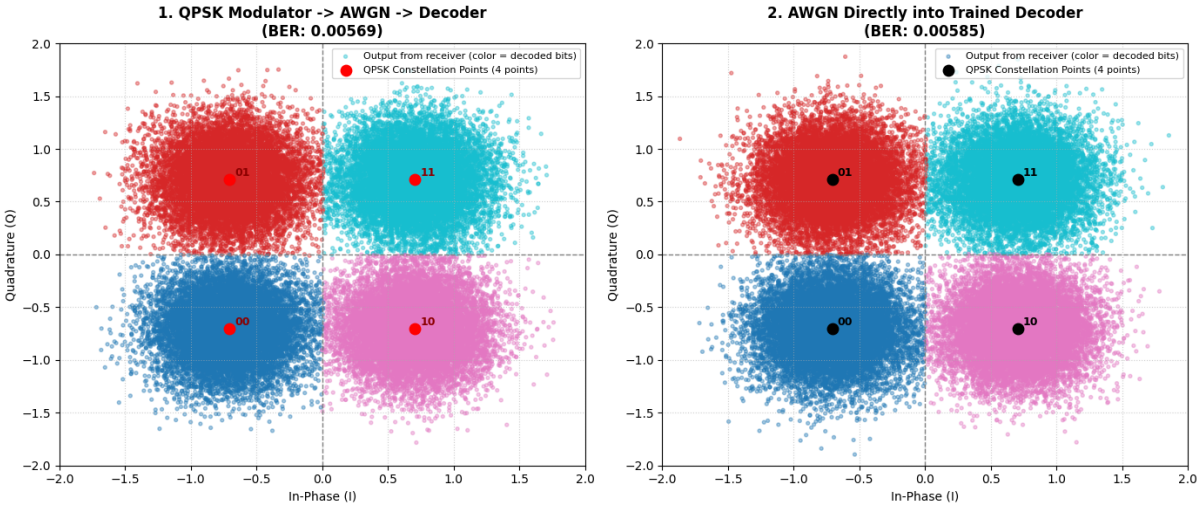


Figure 1: Received I/Q samples at $E_b/N_0 = 5$ dB. Left: QPSK modulator followed by AWGN and the learned decoder. Right: AWGN applied directly to the codewords and passed to the same decoder. Colours indicate the decoded bit pair; red and black markers show the true constellation points.

4.2 QPSK BER Across Eb/N0

Figure 2 plots the learned decoder's BER against the QPSK theory curve over $E_b/N_0 = 0$ to 10 dB. Table 2 lists the theoretical values at each point, together with the learned BER values read from the figure.

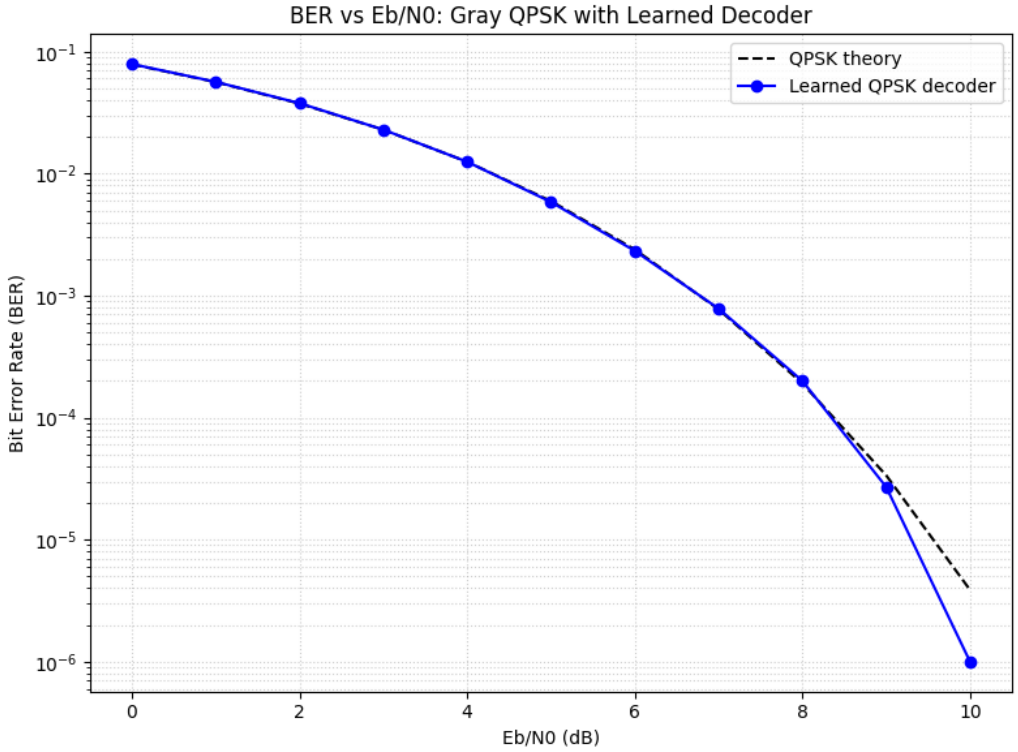


Figure 2: BER versus E_b/N_0 for Gray QPSK. The dashed line is the closed-form QPSK theory; the solid line with markers is the BER of the learned decoder, with 10^6 test bits per point.

Table 2: Theoretical and learned QPSK BER across the E_b/N_0 sweep. Theory values are computed from the closed form. Learned values are approximate readings from Figure 2.

E_b/N_0 (dB)	Theory BER	Learned BER (approx.)
0	7.9×10^{-2}	7.8×10^{-2}
1	5.7×10^{-2}	5.7×10^{-2}
2	3.7×10^{-2}	3.8×10^{-2}
3	2.3×10^{-2}	2.3×10^{-2}
4	1.2×10^{-2}	1.25×10^{-2}
5	5.9×10^{-3}	5.7–5.9×10^{-3}
6	2.4×10^{-3}	2.3×10^{-3}
7	7.7×10^{-4}	7.8×10^{-4}
8	1.9×10^{-4}	2.0×10^{-4}
9	3.4×10^{-5}	2.7×10^{-5}
10	3.9×10^{-6}	1.0×10^{-6}

Across the range 0 to 8 dB, the learned decoder agrees with theory to within the visual resolution of the plot, and the curves overlap almost exactly over 0 to 7 dB. At 9 and 10 dB, the learned points fall slightly below the theory curve. At these operating points the expected error counts in a 10^6-bit test set are small: about 34 errors at 9 dB and about 4 at 10 dB. The

observed value at 10 dB corresponds to roughly one error, so the deviation is within Monte Carlo noise rather than a systematic failure of the decoder. Reliable confirmation at high E_b/N_0 would require substantially more test bits, on the order of 10^8 for a 10 dB point.

4.3 Learned 16-ary Constellation

Figure 3 shows the 16 learned constellation points at the training SNR of 7 dB. The constellation is clearly non-uniform and does not resemble a standard 16-PSK or 16-QAM layout. Reading the approximate coordinates from the figure, the points organize into three groups:

- an outer ring of nine points at radius approximately 1.75 (labels 0, 2, 3, 4, 9, 11, 12, 13, 15);
- an inner ring of six points at radius approximately 0.83 (labels 1, 5, 6, 8, 10, 14);
- a single point at the origin (label 7), located at approximately $(0.02, 0.03)$.

The outer ring is not evenly spaced and the inner ring contains only six points, so the arrangement is an optimized packing rather than a symmetric design. The origin point carries very little energy, which is a notable feature of the solution and is discussed in Section 5.

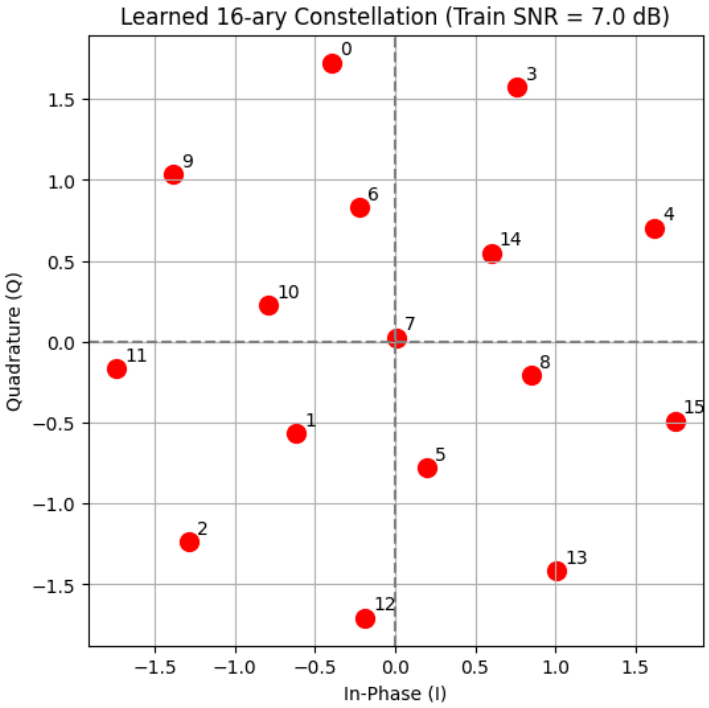


Figure 3: Learned 16-ary two-dimensional constellation after training at SNR = 7 dB. Labels give the message index.

The symbol error rate versus SNR was computed in the same script, using 10^4 random messages per SNR point over $\{0, 2, \ldots, 12\}$ dB. Table 3 is a template for these values; the numerical SER values are printed to the console by the script and should be transcribed here from that output.

Table 3: Symbol error rate of the learned 16-ary autoencoder versus SNR. Values to be transcribed from the console output of the evaluation script.

SNR (dB)	SER
0	0.59760
2	0.50200
4	0.37680
6	0.25080
8	0.14240
10	0.06170
12	0.01840

5 Discussion

5.1 Learned QPSK Receivers Match Theory

The QPSK results show that a small MLP, trained only on noisy samples and their transmitted bits, reproduces the QPSK BER curve across roughly 8 orders of magnitude. The decision regions the network learns are the expected quadrant boundaries. The receiver is not told the optimal decision rule, yet it discovers it from data. Because the training objective is per-bit binary cross-entropy, the learned decoder is effectively estimating the bit-wise posterior and thresholding it. For equal priors and symmetric noise, this reduces to the sign test that defines optimal Gray-QPSK detection.

5.2 Equivalence of the Two Setups

Setup 1 and Setup 2 are not independent experiments. They use the same channel and decoder and differ only in how the transmitted symbols are generated. Their agreement is therefore a consistency check on the code rather than a new scientific result. Its main value is to confirm that indexing the codewords by message does not introduce a bug in the bit-to-symbol mapping.

5.3 The Autoencoder Discovers a Non-Uniform Constellation

The 16-ary autoencoder does not learn a standard 16-point constellation. Instead it produces a two-ring arrangement with a central point. Several interpretations are plausible. An outer ring with more points than the inner ring places more of the symbols at high energy, which can improve the separation between points at the cost of average power. The solution also appears to use the origin as an additional symbol. Because the power normalization enforces a fixed average energy across the constellation, the optimizer is free to trade energy between points. The origin point is energy-inefficient in isolation, but it may be useful as a separator for the decoder's decision regions. A full analysis would compare the learned constellation with 16-QAM and 16-PSK at matched average power and matched SER, which is left for future work.

5.4 Normalization Convention and Effective SNR

A subtle point in the autoencoder code affects how the nominal SNR should be interpreted. The power normalization enforces a mean squared norm of $n_{\text{dim}} = 2$ per symbol, not 1. The noise variance is set to $\sigma^2 = 1/(2\,\text{SNR})$ per real dimension, so the noise power per symbol is $N_0 = 1/\text{SNR}$. The effective ratio of symbol energy to noise density is therefore

$$\frac{E_s}{N_0} = \frac{2}{1/\text{SNR}} = 2\,\text{SNR}, \tag{6}$$

which is 3 dB higher than the nominal SNR used in the labels and sweep. This does not invalidate the qualitative results, but it means the SNR axis of the SER curve should not be compared directly to the E_b/N_0 axis of the QPSK results, nor to textbook values for 16-QAM, without this correction. A cleaner design would normalize to unit energy and define σ^2 from the target E_s/N_0 explicitly.

5.5 Limitations

Several limitations should be noted:

- **Monte Carlo noise.** The high-E_b/N_0 points in the sweep use few errors, so their values are imprecise. The reported 10 dB value is not a reliable estimate.
- **Single seed.** Each study uses one random seed (`torch.manual_seed(0)`). No confidence intervals across seeds are reported, so the variability due to initialization and training is not quantified.
- **Approximate learned values in the sweep table.** The learned BER values in Table 2 are read from the plot. Exact values should be taken from the printed output of the script.
- **No baseline for the autoencoder.** The 16-ary result is not compared to a conventional constellation or to the theoretical capacity limit.
- **Fixed training SNR.** The autoencoder is trained at a single SNR. Its constellation may not be optimal across the full SER sweep.
- **Decoder capacity and training budget.** The decoders are small and trained for a fixed number of steps. Convergence was not checked against a longer schedule.
- **Ideal channel.** Only AWGN is modelled, with perfect synchronization and perfect knowledge of the noise level in the receiver.

6 Conclusion

This project demonstrated two things. First, a small neural network trained only on noisy QPSK samples learns a receiver whose BER matches the closed-form Gray-QPSK expression over 0 to 8 dB, with deviations at higher E_b/N_0 that are explained by limited test-set size. At 5 dB, the two evaluation setups give BERs of 5.69×10^{-3} and 5.85×10^{-3}, bracketing the theoretical value of 5.94×10^{-3}. Second, a 16-ary autoencoder trained at 7 dB converges to a non-uniform constellation with two rings and a central point, which suggests that gradient-based training can find signal designs that depart substantially from textbook layouts.

Future Work

- Run the sweep with 10^7 or more test bits at high E_b/N_0 to resolve the tail of the BER curve.
- Repeat each study over several random seeds and report means and confidence intervals.
- Compare the learned 16-point constellation against 16-QAM and 16-PSK at matched average power, plotting SER on a common axis.
- Train the autoencoder across a range of SNRs or use a curriculum, and assess whether a single learned constellation generalizes.
- Extend the channel model to include phase offset, fading, or carrier frequency offset, where closed-form receivers are not available and the learned approach is most useful.

A QPSK Code at 5 dB

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np
torch.manual_seed(0)
num_bits = 100000
ebn0_db = 5.0
ebn0_linear = 10.0 ** (ebn0_db / 10.0)
sigma = 0.5 / np.sqrt(ebn0_linear)
train_steps = 5000
batch_size = 1024
lr = 1e-3
msgs = torch.tensor([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=torch.float32)
def encode_qpsk(bits):
    I = (2.0 * bits[..., 0] - 1.0) / np.sqrt(2.0)
    Q = (2.0 * bits[..., 1] - 1.0) / np.sqrt(2.0)
    return torch.stack([I, Q], dim=-1)
decoder = nn.Sequential(nn.Linear(2, 64), nn.ReLU(), nn.Linear(64, 2))
def awgn(x):
    return x + torch.randn_like(x) * sigma
optimizer = torch.optim.Adam(decoder.parameters(), lr=lr)
loss_fn = nn.BCEWithLogitsLoss()
for step in range(train_steps):
    bits = torch.randint(0, 2, (batch_size, 2)).float()
    loss = loss_fn(decoder(awgn(encode_qpsk(bits))), bits)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 1000 == 0 or step == train_steps - 1:
        print(f"step {step:5d}  loss {loss.item():.4f}")
with torch.no_grad():
    bits_test = torch.randint(0, 2, (num_bits // 2, 2)).float()
    y_test = awgn(encode_qpsk(bits_test))
    ber_setup1 = ((decoder(y_test) > 0).float() != bits_test).float().mean().
        item()
with torch.no_grad():
    codewords = encode_qpsk(msgs)                          # (4, 2) clean QPSK
        symbols
    msg_idx = torch.randint(0, 4, (num_bits // 2,))        # random 2-bit
        messages
    bits_direct = msgs[msg_idx]
    y_direct = awgn(codewords[msg_idx])                    # AWGN directly on
        signal
    bits_direct_hat = (decoder(y_direct) > 0).float()
    ber_setup2 = (bits_direct_hat != bits_direct).float().mean().item()
    decoded_msg = (bits_direct_hat * torch.tensor([2, 1.])).sum(-1).long()
print("\n--- Simulation Results (QPSK) ---")
print(f"Setup 1 BER: {ber_setup1:.6f}")
print(f"Setup 2 BER: {ber_setup2:.6f}")
cw = codewords.numpy()
y1 = y_test.numpy()
y_np = y_direct.numpy()
dec_np = decoded_msg.numpy()
with torch.no_grad():
    dec_idx = ((decoder(y_test) > 0).float() @ torch.tensor([2.0, 1.0])).long().
        numpy()
msg_labels = [f"{int(m[0])}{int(m[1])}" for m in msgs]
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
plt.scatter(y1[:, 0], y1[:, 1], c=dec_idx, cmap='tab10', alpha=0.4, s=8,
            label='Output from receiver (color = decoded bits)')
```

```
plt.scatter(cw[:, 0], cw[:, 1], color='red', s=80, zorder=5,
            label='QPSK Constellation Points (4 points)')
for x, y, label in zip(cw[:, 0], cw[:, 1], msg_labels):
    plt.text(x + 0.04, y + 0.04, label, fontsize=9, fontweight='bold', color='
        darkred', zorder=6)
plt.axhline(0, color='grey', linestyle='--', linewidth=1)
plt.axvline(0, color='grey', linestyle='--', linewidth=1)
plt.xlim(-2, 2); plt.ylim(-2, 2)
plt.xlabel('In-Phase (I)'); plt.ylabel('Quadrature (Q)')
plt.title(f'1. QPSK Modulator -> AWGN -> Decoder\n(BER: {ber_setup1:.5f})',
    fontweight='bold')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper right', fontsize=8)
plt.subplot(1, 2, 2)
plt.scatter(y_np[:, 0], y_np[:, 1], c=dec_np, cmap='tab10', alpha=0.4, s=8,
            label='Output from receiver (color = decoded bits)')
plt.scatter(cw[:, 0], cw[:, 1], color='black', s=80, zorder=5,
            label='QPSK Constellation Points (4 points)')
for x, y, label in zip(cw[:, 0], cw[:, 1], msg_labels):
    plt.text(x + 0.04, y + 0.04, label, fontsize=9, fontweight='bold', color='
        black', zorder=6)
plt.axhline(0, color='grey', linestyle='--', linewidth=1)
plt.axvline(0, color='grey', linestyle='--', linewidth=1)
plt.xlim(-2, 2); plt.ylim(-2, 2)
plt.xlabel('In-Phase (I)'); plt.ylabel('Quadrature (Q)')
plt.title(f'2. AWGN Directly into Trained Decoder\n(BER: {ber_setup2:.5f})',
    fontweight='bold')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper right', fontsize=8)
plt.tight_layout()
plt.show()
```

B BER Code: Learned QPSK Receiver over Eb/N0

```
import math
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np
torch.manual_seed(0)
ebn0_db_list = np.arange(0, 11, 1)      # Eb/N0 sweep in dB
train_steps = 5000
batch_size = 1024
lr = 1e-3
num_test_bits = 1_000_000
inv_sqrt2 = 1.0 / np.sqrt(2.0)

def encode_qpsk(bits):
    I = (2.0 * bits[..., 0] - 1.0) * inv_sqrt2
    Q = (2.0 * bits[..., 1] - 1.0) * inv_sqrt2
    return torch.stack([I, Q], dim=-1)
def awgn(x, sigma):
    return x + torch.randn_like(x) * sigma
ber_learned = []
ber_theory = []
for ebn0_db in ebn0_db_list:
    snr = 10.0 ** (ebn0_db / 10.0)
    sigma = 0.5 / np.sqrt(snr)
    decoder = nn.Sequential(nn.Linear(2, 64), nn.ReLU(), nn.Linear(64, 2))
    optimizer = torch.optim.Adam(decoder.parameters(), lr=lr)
    loss_fn = nn.BCEWithLogitsLoss()
    for step in range(train_steps):
        bits = torch.randint(0, 2, (batch_size, 2)).float()
```

```
        loss = loss_fn(decoder(awgn(encode_qpsk(bits), sigma)), bits)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        bits_test = torch.randint(0, 2, (num_test_bits // 2, 2)).float()
        y = awgn(encode_qpsk(bits_test), sigma)
        ber = ((decoder(y) > 0).float() != bits_test).float().mean().item()
    ber_learned.append(max(ber, 1e-7))
    ber_theory.append(0.5 * math.erfc(math.sqrt(snr)))   # Q(sqrt(2 Eb/N0))
    print(f"Eb/N0 {ebn0_db:4.1f} dB | learned BER {ber:.2e} | QPSK theory {
        ber_theory[-1]:.2e}")
plt.figure(figsize=(8, 6))
plt.semilogy(ebn0_db_list, ber_theory, 'k--', label='QPSK theory')
plt.semilogy(ebn0_db_list, ber_learned, 'o-', color='blue', label='Learned QPSK
    decoder')
plt.grid(True, which='both', linestyle=':', alpha=0.6)
plt.xlabel('Eb/N0 (dB)')
plt.ylabel('Bit Error Rate (BER)')
plt.title('BER vs Eb/N0: Gray QPSK with Learned Decoder')
plt.legend()
plt.tight_layout()
plt.show()
```

C Autoencoder Code: 16-ary Learned Constellation

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
M = 16
n_dim = 2
batch_size = 512
epochs = 200
steps_per_epoch = 100
learning_rate = 0.01
train_snr_db = 7.0
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
w_enc1 = nn.Parameter(torch.randn(32, M, device=device) * np.sqrt(2.0 / M))
b_enc1 = nn.Parameter(torch.zeros(32, device=device))
w_enc2 = nn.Parameter(torch.randn(n_dim, 32, device=device) * np.sqrt(2.0 / 32))
b_enc2 = nn.Parameter(torch.zeros(n_dim, device=device))
w_dec1 = nn.Parameter(torch.randn(32, n_dim, device=device) * np.sqrt(2.0 /
    n_dim))
b_dec1 = nn.Parameter(torch.zeros(32, device=device))
w_dec2 = nn.Parameter(torch.randn(M, 32, device=device) * np.sqrt(2.0 / 32))
b_dec2 = nn.Parameter(torch.zeros(M, device=device))
parameters = [w_enc1, b_enc1, w_enc2, b_enc2, w_dec1, b_dec1, w_dec2, b_dec2]
optimizer = optim.Adam(parameters, lr=learning_rate)
criterion = nn.CrossEntropyLoss()
def encoder(x):
    h = F.relu(F.linear(x, w_enc1, b_enc1))
    return F.linear(h, w_enc2, b_enc2)
def decoder(x):
    h = F.relu(F.linear(x, w_dec1, b_dec1))
    return F.linear(h, w_dec2, b_dec2)
def normalize_power(x):
    return torch.sqrt(torch.tensor(n_dim, dtype=torch.float32, device=x.device))
        * \
           (x / torch.sqrt(torch.mean(torch.sum(x**2, dim=1, keepdim=True))))
def awgn_channel(x, snr_db):
```

```
    snr_linear = 10.0 ** (snr_db / 10.0)
    noise_variance = 1.0 / (2.0 * snr_linear)
    noise = torch.sqrt(torch.tensor(noise_variance, dtype=torch.float32, device=
        x.device)) * torch.randn_like(x)
    return x + noise
def forward(x, snr_db):
    encoded = encoder(x)
    normalized = normalize_power(encoded)
    received = awgn_channel(normalized, snr_db)
    decoded = decoder(received)
    return decoded, normalized
print("Training Autoencoder...")
for epoch in range(epochs):
    epoch_loss = 0.0
    for _ in range(steps_per_epoch):
        messages = torch.randint(0, M, (batch_size,)).to(device)
        one_hot = torch.eye(M, device=device)[messages]
        optimizer.zero_grad()
        outputs, _ = forward(one_hot, train_snr_db)
        loss = criterion(outputs, messages)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    if (epoch + 1) % 25 == 0:
        print(f"Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss/steps_per_epoch:.4
            f}")
with torch.no_grad():
    all_messages = torch.eye(M, device=device)
    _, constellation = forward(all_messages, train_snr_db)
    constellation_np = constellation.cpu().numpy()
plt.figure(figsize=(6, 6))
plt.scatter(constellation_np[:, 0], constellation_np[:, 1], color='red', s=100)
for i in range(M):
    plt.annotate(str(i), (constellation_np[i, 0] + 0.05, constellation_np[i, 1]
        + 0.05))
plt.axhline(0, color='grey', linestyle='--')
plt.axvline(0, color='grey', linestyle='--')
plt.title(f"Learned 16-ary Constellation (Train SNR = {train_snr_db} dB)")
plt.xlabel("In-Phase (I)")
plt.ylabel("Quadrature (Q)")
plt.grid(True)
plt.show()
snr_range = np.arange(0, 13, 2)
ser_results = []
print("\nEvaluating Symbol Error Rate (SER) vs SNR...")
with torch.no_grad():
    for snr in snr_range:
        errors = 0
        total_samples = 10000
        for _ in range(total_samples // batch_size):
            messages = torch.randint(0, M, (batch_size,)).to(device)
            one_hot = torch.eye(M, device=device)[messages]
            outputs, _ = forward(one_hot, snr)
            predictions = torch.argmax(outputs, dim=1)
            errors += torch.sum(predictions != messages).item()
        ser = errors / total_samples
        ser_results.append(ser)
        print(f"SNR: {snr:2d} dB | SER: {ser:.5f}")
```